Prediction of Stress-Relief Gaming Behavior Among Students Using Naive Bayes

Problem Statement:

Students often engage in gaming as a way to cope with academic stress, but gaming behavior varies based on personal, academic, and lifestyle factors. This project aims to predict whether students play games for stress relief using the Naive Bayes classification algorithm by analyzing their gaming habits, academic performance, daily routines, and emotional responses. The model helps identify patterns linking gaming behavior and stress management.

In [ ]:

Steps to be followed

1. Importing Required Libraries
2. Load Dataset
3. Data Preprocessing
4. Checking Null values
5. Data Wrangling (Filling Missing values)
6. Data Visualization
7. Feature Selection (x,y)
8. Train-Test Split
9. Bernoulli Naive Bayes Model Training and Prediction
10. Model Evaluation (Accuracy Score,Classification Report)
11. Confusion Matrix
12. ROC-AUC Score and ROC-AUC Curve
13. GridSearchCV and RandomizedSearchCV
14. Improving Model Performance with Advanced Algorithms

Importing Required Libraries

In [2]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
import plotly.express as px
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split, GridSearchCV ,cross_val_
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder,
from sklearn.metrics import classification_report ,confusion_matrix,accuracy_s
import warnings
warnings.filterwarnings('ignore')
```

Loading the Dataset

In [3]:
```
df=pd.read_csv(r"C:\Users\HP\Downloads\Student Gaming and Academic Behaviour.c
df
```

Out[3]:

	What is your age?	Current educational position?	Gender?	Your current CGPA?	Your Higher Secondary School(H. SC) or A level or equivalent result?	At what age you had started playing games?	Do you play games on mobile or pc?	When you go to sleep?
0	21.0	Bachelor Level	Male	4.00	5.00	10	Mobile, PC	2.0
1	21.0	Bachelor Level	Male	3.00	5.00	11	Mobile	2.0
2	21.0	NaN	Male	3.75	5.00	12	Mobile, PC	1.0
3	20.0	Bachelor Level	Male	3.50	5.00	18	Mobile	0.0
4	25.0	Bachelor Level	NaN	2.00	5.00	14	Mobile	NaN
...	...	...	...	...	...	...	...	...
984	20.0	Bachelor Level	Female	3.20	3.24	21	PC	NaN
985	24.0	NaN	Male	2.72	4.41	19	Mobile	1.0
986	21.0	NaN	Female	3.55	5.00	20	Mobile	NaN
987	22.0	Bachelor Level	Male	3.83	3.36	19	PC	23.0
988	24.0	Master's Level	Male	3.97	5.00	21	PC	NaN

989 rows × 19 columns

In [ ]:

Data Preprocessing

In [4]:
```
df.shape
```

Out[4]:
```
(989, 19)
```

In [5]:
```
df.head()
```

Out[5]:

	What is your age?	Current educational position?	Gender?	Your current CGPA?	Your Higher Secondary School(H. SC) or A level or equivalent result?	At what age you had started playing games?	Do you play games on mobile or pc?	When you go to sleep?	re
0	21.0	Bachelor Level	Male	4.00	5.0	10	Mobile, PC	2.0	
1	21.0	Bachelor Level	Male	3.00	5.0	11	Mobile	2.0	
2	21.0	NaN	Male	3.75	5.0	12	Mobile, PC	1.0	
3	20.0	Bachelor Level	Male	3.50	5.0	18	Mobile	0.0	
4	25.0	Bachelor Level	NaN	2.00	5.0	14	Mobile	NaN	

In [6]:
```
df.tail()
```

Out[6]:

	What is your age?	Current educational position?	Gender?	Your current CGPA?	Your Higher Secondary School(H. SC) or A level or equivalent result?	At what age you had started playing games?	Do you play games on mobile or pc?	When you go to sleep?
984	20.0	Bachelor Level	Female	3.20	3.24	21	PC	NaN
985	24.0	NaN	Male	2.72	4.41	19	Mobile	1.0
986	21.0	NaN	Female	3.55	5.00	20	Mobile	NaN
987	22.0	Bachelor Level	Male	3.83	3.36	19	PC	23.0
988	24.0	Master's Level	Male	3.97	5.00	21	PC	NaN

In [ ]:

In [ ]:

In [7]:
```
df.sample()
```

Out[7]:

	What is your age?	Current educational position?	Gender?	Your current CGPA?	Your Higher Secondary School(H. SC) or A level or equivalent result?	At what age you had started playing games?	Do you play games on mobile or pc?	When you go to sleep?
895	25.0	Master's Level	Female	NaN	3.98	21	PC	0.0

In [8]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 989 entries, 0 to 988
Data columns (total 19 columns):
 #   Column                                                                  Non-
Null Count  Dtype
---  ------
--------------  -----
 0   What is your age?                                                       954
non-null    float64
 1   Current educational position?                                           751
non-null    object
 2   Gender?                                                                 852
non-null    object
 3   Your  current CGPA?                                                     958
non-null    float64
 4   Your Higher Secondary School(H. SC) or A level or equivalent result?  989
non-null    float64
 5   At what age you had started playing games?                              989
non-null    int64
 6   Do you play games on mobile or pc?                                      989
non-null    object
 7   When you go to sleep?                                                   823
non-null    float64
 8   Do you attend your morning class regularly?                             664
non-null    object
 9   The average time you spend playing games?                               781
non-null    float64
 10  Which type of game you addict more?                                     719
non-null    object
 11  Do you read newspaper?                                                  989
non-null    object
 12  How many time you spend with family and friend?                         906
non-null    object
 13  How you feel when you can not play game in whole day?                   712
non-null    object
 14  How you feel to complete game level?                                    541
non-null    object
 15  If you did not finish games last level what is your feeling?           989
non-null    object
 16  Do you feel Fatigue                                                     989
non-null    object
 17  Do you play games for stress relief?                                    989
non-null    object
 18  Are you wearing glasses?                                                673
non-null    object
dtypes: float64(5), int64(1), object(13)
memory usage: 146.9+ KB
```

In [9]:
```
df.describe()
```

Out[9]:

	What is your age?	Your current CGPA?	Your Higher Secondary School(H. SC) or A level or equivalent result?	At what age you had started playing games?	When you go to sleep?	The average time you spend playing games?
count	954.000000	958.000000	989.000000	989.000000	823.000000	781.000000
mean	22.995807	3.137526	4.155399	19.528817	9.664642	3.777209
std	3.166586	0.561157	0.657749	2.570943	10.529038	1.596966
min	18.000000	2.000000	2.000000	3.000000	0.000000	0.000000
25%	20.250000	2.842500	3.810000	18.000000	1.000000	2.000000
50%	23.000000	3.030000	4.100000	20.000000	2.000000	4.000000
75%	26.000000	3.580000	4.730000	21.000000	22.000000	5.000000
max	30.000000	4.000000	5.000000	24.000000	23.000000	8.000000

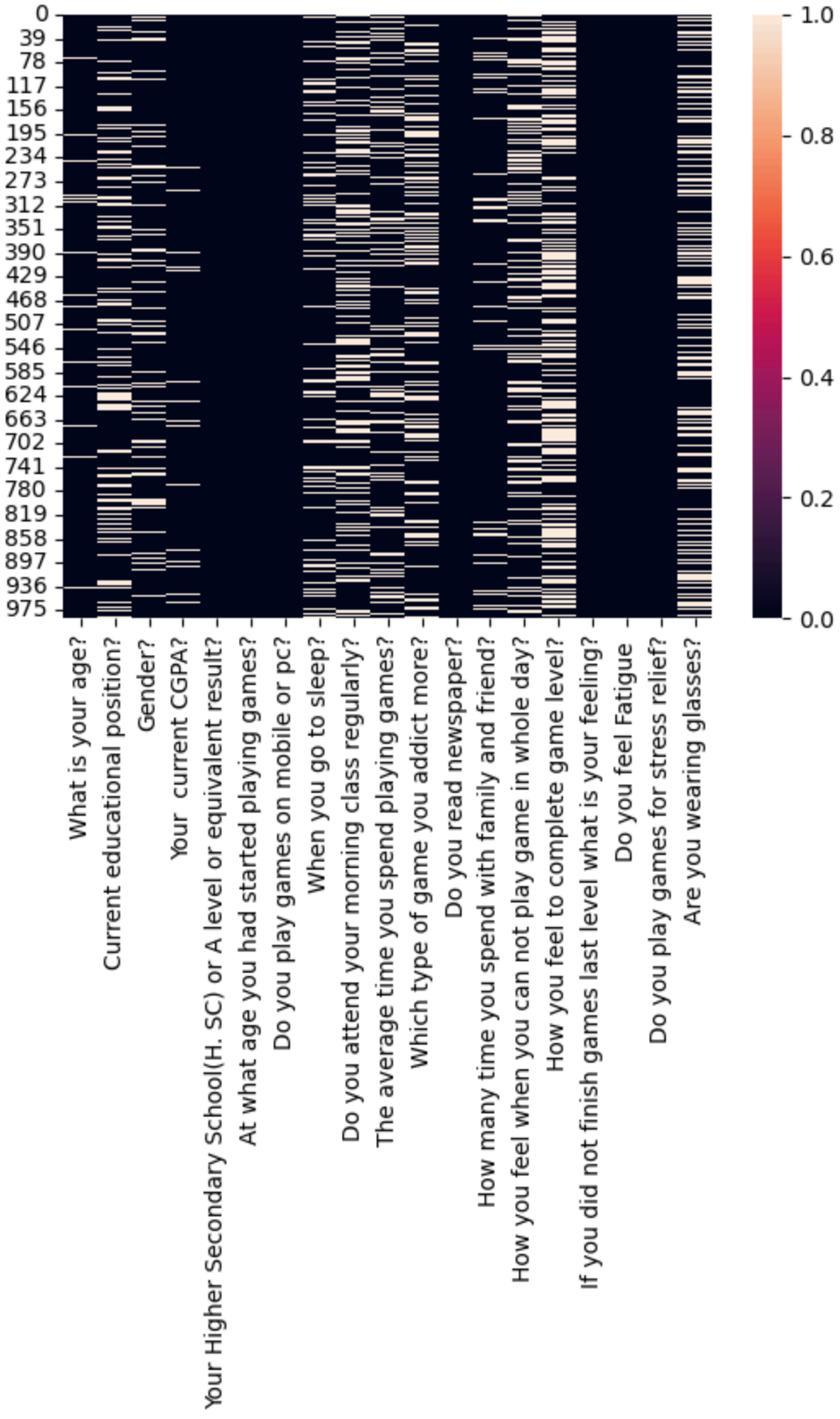
Checking Null Values

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
What is your age?                                                        35
Current educational position?                                           238
Gender?                                                                 137
Your  current CGPA?                                                      31
Your Higher Secondary School(H. SC) or A level or equivalent result?      0
At what age you had started playing games?                                0
Do you play games on mobile or pc?                                        0
When you go to sleep?                                                   166
Do you attend your morning class regularly?                             325
The average time you spend playing games?                               208
Which type of game you addict more?                                     270
Do you read newspaper?                                                    0
How many time you spend with family and friend?                          83
How you feel when you can not play game in whole day?                   277
How you feel to complete game level?                                    448
If you did not finish games last level what is your feeling?             0
Do you feel Fatigue                                                       0
Do you play games for stress relief?                                      0
Are you wearing glasses?                                                316
dtype: int64
```

In [11]:
```
sns.heatmap(df.isnull())
```

Out[11]:
```
<Axes: >
```



In [12]:
```
df.columns
```

Out[12]:
```
Index(['What is your age?', 'Current educational position?', 'Gender?',
       'Your  current CGPA?',
       'Your Higher Secondary School(H. SC) or A level or equivalent resul
t?',
       'At what age you had started playing games?',
       'Do you play games on mobile or pc?', 'When you go to sleep?',
       'Do you attend your morning class regularly?',
       'The average time you spend playing games?',
       'Which type of game you addict more?', 'Do you read newspaper?',
       'How many time you spend with family and friend?',
       'How you feel when you can not play game in whole day?',
       'How you feel to complete game level?',
       'If you did not finish games last level what is your feeling?',
       'Do you feel Fatigue ', 'Do you play games for stress relief?',
       'Are you wearing glasses?'],
      dtype='object')
```

Data Wrangling (Filling Null/Missing Values)

In [ ]:
```
```

In [13]:
```
num_cols=['What is your age?','Your  current CGPA?','When you go to sleep?',
          'The average time you spend playing games?']
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

In [14]:
```
cat_cols=['Current educational position?', 'Gender?', 'Do you attend your morn
          ,'Which type of game you addict more?','How many time you spend with
       'How you feel when you can not play game in whole day?','How you feel t
          'Are you wearing glasses?']
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

In [15]:
```
df.columns = df.columns.str.strip()
```

In [16]:
```
df.isnull().sum()
```

Out[16]:
```
What is your age?                                                    0
Current educational position?                                        0
Gender?                                                              0
Your  current CGPA?                                                  0
Your Higher Secondary School(H. SC) or A level or equivalent result?    0
At what age you had started playing games?                           0
Do you play games on mobile or pc?                                   0
When you go to sleep?                                                0
Do you attend your morning class regularly?                          0
The average time you spend playing games?                            0
Which type of game you addict more?                                  0
Do you read newspaper?                                               0
How many time you spend with family and friend?                      0
How you feel when you can not play game in whole day?                0
How you feel to complete game level?                                 0
If you did not finish games last level what is your feeling?         0
Do you feel Fatigue                                                  0
Do you play games for stress relief?                                 0
Are you wearing glasses?                                             0
dtype: int64
```

In [17]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 989 entries, 0 to 988
Data columns (total 19 columns):
 #   Column                                                                Non-
Null Count  Dtype
---  ------
--------------  -----
 0   What is your age?                                                     989
non-null    float64
 1   Current educational position?                                         989
non-null    object
 2   Gender?                                                               989
non-null    object
 3   Your  current CGPA?                                                   989
non-null    float64
 4   Your Higher Secondary School(H. SC) or A level or equivalent result?  989
non-null    float64
 5   At what age you had started playing games?                            989
non-null    int64
 6   Do you play games on mobile or pc?                                    989
non-null    object
 7   When you go to sleep?                                                 989
non-null    float64
 8   Do you attend your morning class regularly?                           989
non-null    object
 9   The average time you spend playing games?                             989
non-null    float64
 10  Which type of game you addict more?                                   989
non-null    object
 11  Do you read newspaper?                                                989
non-null    object
 12  How many time you spend with family and friend?                       989
non-null    object
 13  How you feel when you can not play game in whole day?                 989
non-null    object
 14  How you feel to complete game level?                                  989
non-null    object
 15  If you did not finish games last level what is your feeling?          989
non-null    object
 16  Do you feel Fatigue                                                   989
non-null    object
 17  Do you play games for stress relief?                                  989
non-null    object
 18  Are you wearing glasses?                                              989
non-null    object
dtypes: float64(5), int64(1), object(13)
memory usage: 146.9+ KB
```

Data Visualization

In [18]:
```
df.columns
```

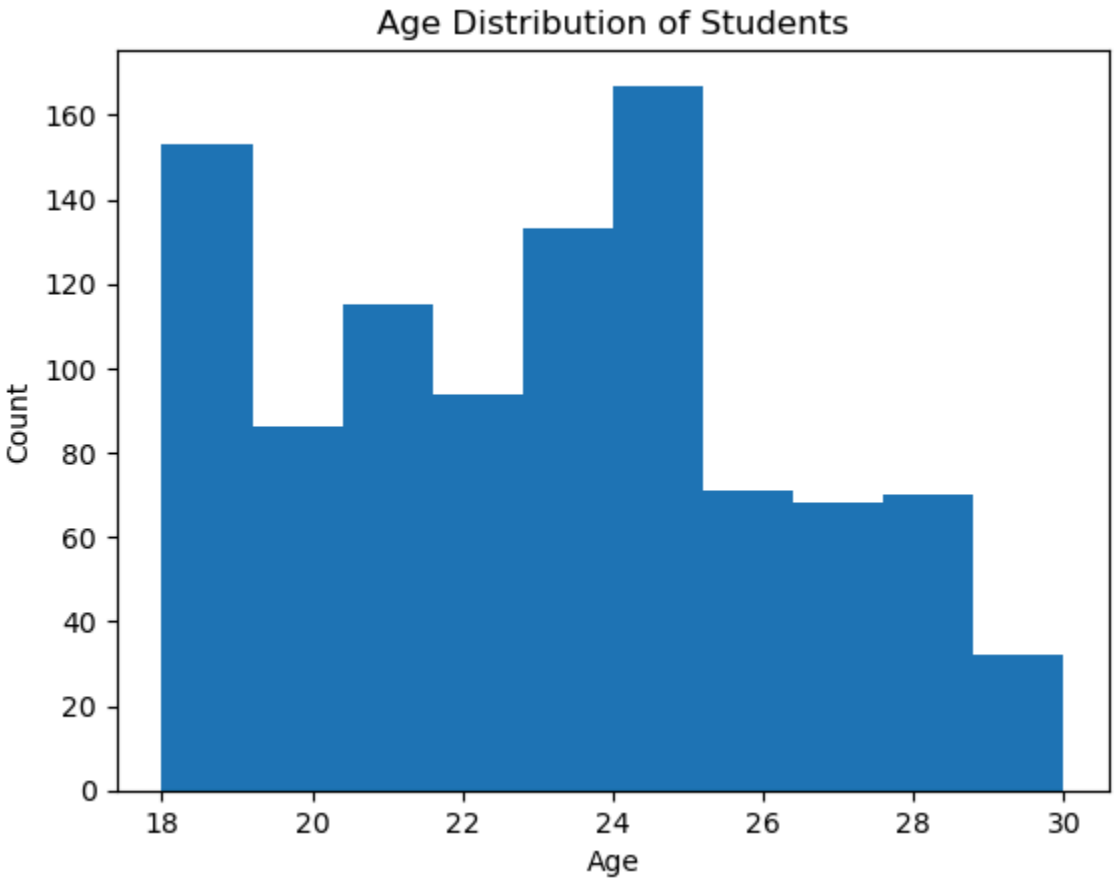
Out[18]:
```
Index(['What is your age?', 'Current educational position?', 'Gender?',
       'Your  current CGPA?',
       'Your Higher Secondary School(H. SC) or A level or equivalent resul
t?',
       'At what age you had started playing games?',
       'Do you play games on mobile or pc?', 'When you go to sleep?',
       'Do you attend your morning class regularly?',
       'The average time you spend playing games?',
       'Which type of game you addict more?', 'Do you read newspaper?',
       'How many time you spend with family and friend?',
       'How you feel when you can not play game in whole day?',
       'How you feel to complete game level?',
       'If you did not finish games last level what is your feeling?',
       'Do you feel Fatigue', 'Do you play games for stress relief?',
       'Are you wearing glasses?'],
      dtype='object')
```

In [19]:
```
plt.figure(figsize=(6,4))
sns.countplot(x='Do you play games for stress relief?', data=df)
plt.title("Target Class Distribution")
plt.show()
```

The target variable is fairly balanced, which helps in building a reliable classification model.

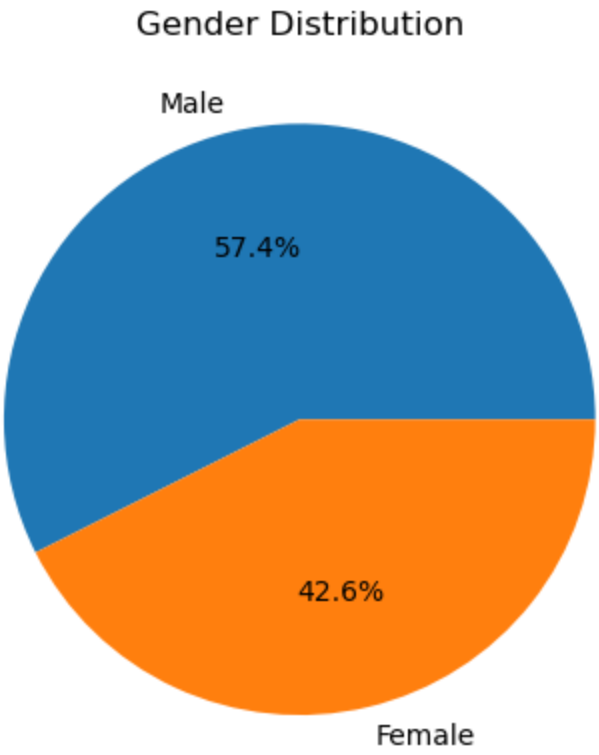
In [20]:
```
plt.figure()
plt.hist(df["What is your age?"].dropna(), bins=10)
plt.title("Age Distribution of Students")
plt.xlabel("Age")
```

```
plt.ylabel("Count")
plt.show()
```



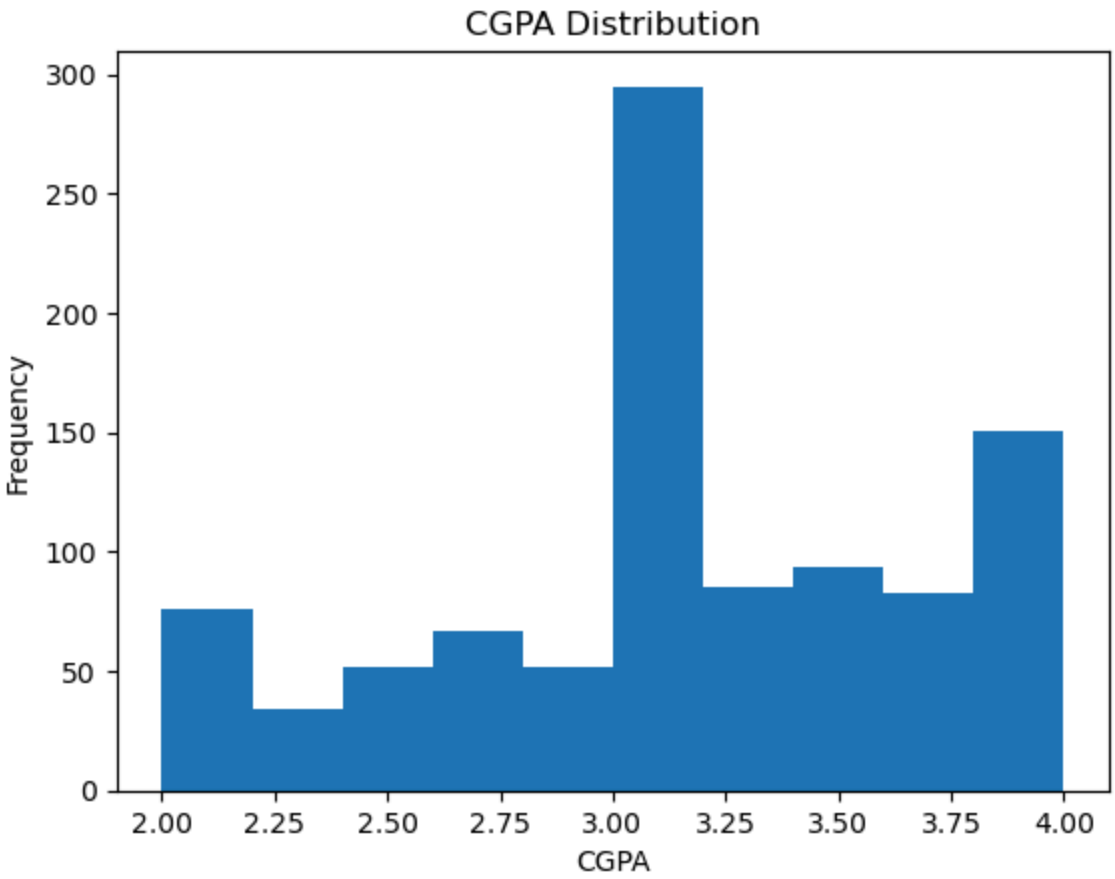
The dataset mainly consists of young students aged between 20 and 25 years.

In [21]:
```
plt.figure()
df["Gender?"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Gender Distribution")
plt.ylabel("")
plt.show()
```



Male participants slightly dominate the dataset, but both genders are reasonably represented.

In [22]:
```
plt.figure()
plt.hist(df["Your  current CGPA?"].dropna(), bins=10)
plt.title("CGPA Distribution")
plt.xlabel("CGPA")
plt.ylabel("Frequency")
plt.show()
```

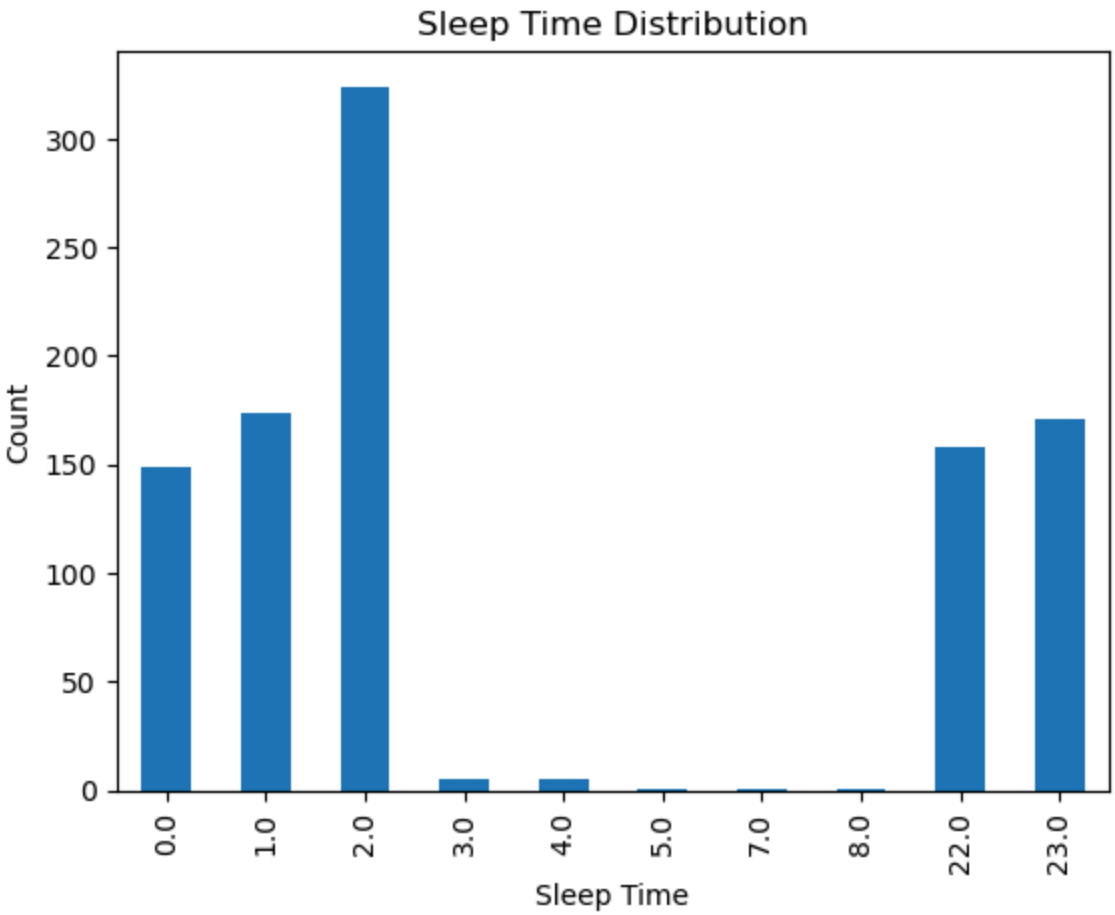


The majority of students maintain a moderate to high CGPA level.

In [23]:
```
fig = px.scatter(
    df,
    x="The average time you spend playing games?",
    y="Your  current CGPA?",
    title="Gaming Hours vs CGPA",
    trendline="ols")
fig.show()
```

Higher gaming hours may slightly reduce academic performance, though the relationship is weak.

In [24]:
```
plt.figure()
df["When you go to sleep?"].value_counts().sort_index().plot(kind="bar")
plt.title("Sleep Time Distribution")
plt.xlabel("Sleep Time")
plt.ylabel("Count")
plt.show()
```



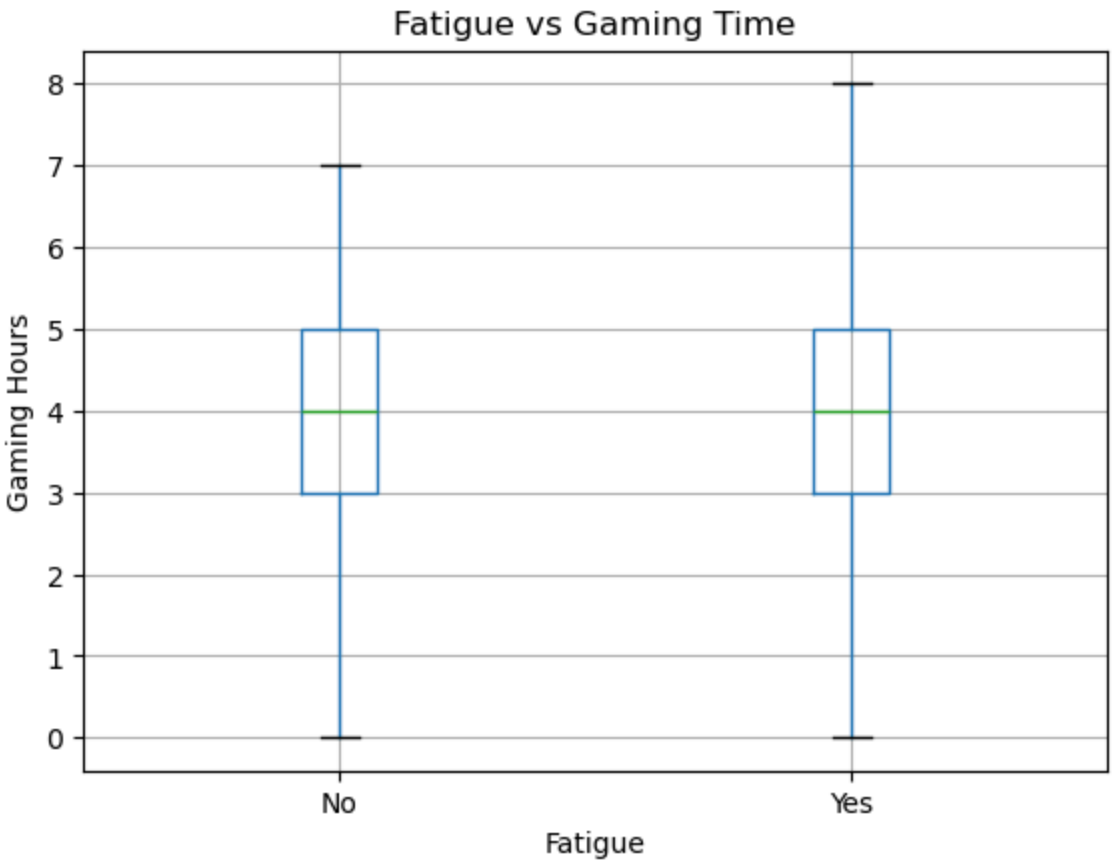
Students commonly sleep late at night, with peak sleeping times between 10 PM and 2 AM.

In [25]:
```
fig = px.box(
    df,
    x="Do you play games for stress relief?",
    y="The average time you spend playing games?",
    title="Stress Relief vs Gaming Time"
)
fig.show()
```

Students who use gaming for stress relief generally spend more time playing games.

In [26]:
```
plt.figure()
df.boxplot(
    column="The average time you spend playing games?",
    by="Do you feel Fatigue")
plt.title("Fatigue vs Gaming Time")
plt.suptitle("")
plt.xlabel("Fatigue")
plt.ylabel("Gaming Hours")
plt.show()
```

```
<Figure size 640x480 with 0 Axes>
```

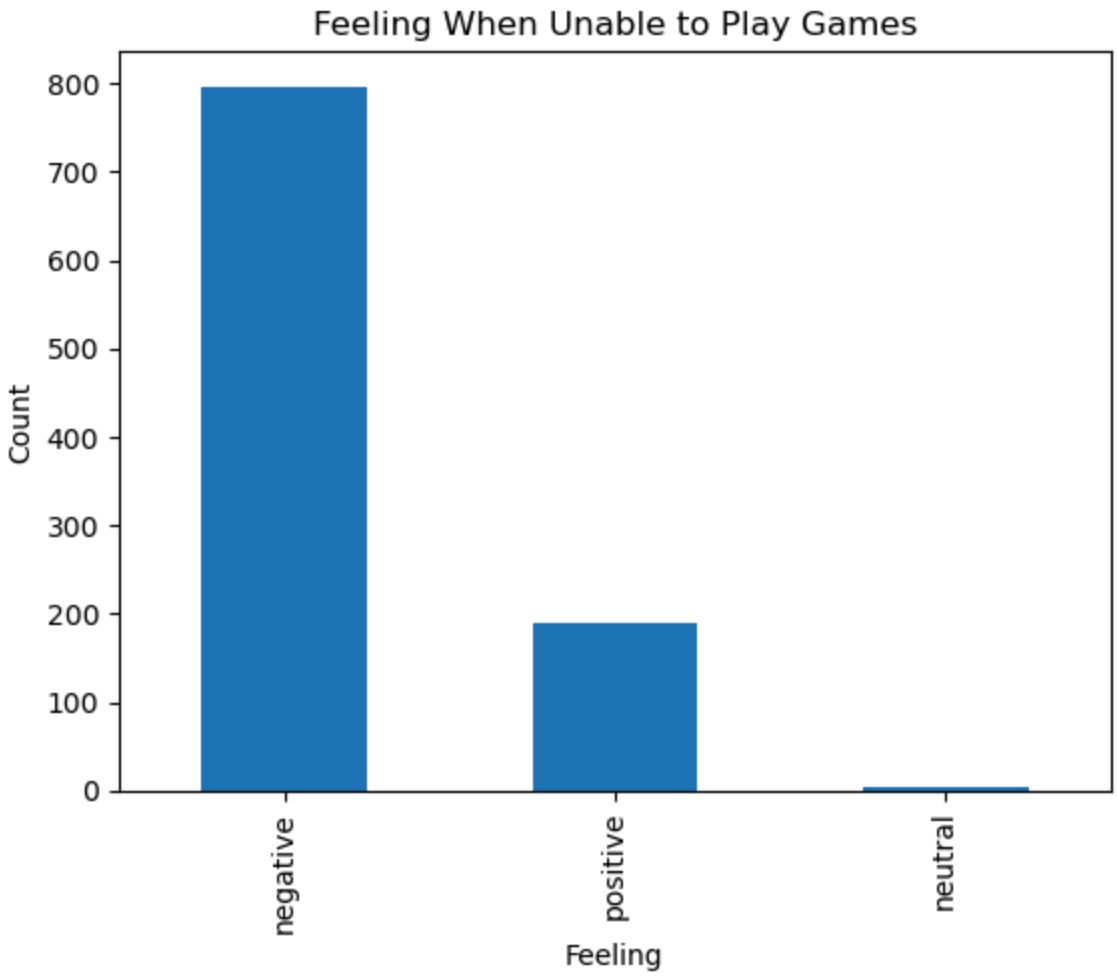


Higher gaming time may increase the likelihood of fatigue among students.

In [27]:
```
fig = px.box(
    df,
    x="How many time you spend with family and friend?",
    y="The average time you spend playing games?",
    title="Family/Friend Time vs Gaming Hours")
fig.show()
```

Students who spend time with family or friends tend to maintain balanced gaming habits.

In [28]:
```
plt.figure()
df["How you feel when you can not play game in whole day?"].value_counts().plo
plt.title("Feeling When Unable to Play Games")
plt.xlabel("Feeling")
plt.ylabel("Count")
plt.show()
```



Many students experience negative emotions when unable to play games, indicating a strong engagement with gaming.

Feature Selection (x,y)

In [29]:
```
y = df['Do you play games for stress relief?']
x = df.drop(columns=['Do you play games for stress relief?'])
```

In [30]:
```
x_encoded = pd.get_dummies(x, drop_first=True)
```

In [ ]:
```

```

Train-Test Split

The dataset is divided into 80% training data and 20% testing data.

In [31]:
```
x_train, x_test, y_train, y_test = train_test_split(x_encoded, y, test_size=0.
```

In [46]:
```
x_train.columns = x_train.columns.str.replace('[^A-Za-z0-9_]', '', regex=True)
x_test.columns = x_test.columns.str.replace('[^A-Za-z0-9_]', '', regex=True)
```

Model Training

In [32]:
```
from sklearn.naive_bayes import BernoulliNB

nb = BernoulliNB()
nb.fit(x_train, y_train)
```

Out[32]:
```
BernoulliNB
BernoulliNB()
```

Model Prediction

In [33]:
```
y_pred = nb.predict(x_test)
```

In [34]:
```
x_train.dtypes[x_train.dtypes == 'object']
```

Out[34]:
```
Series([], dtype: object)
```

Model Evaluation

Accuracy Score

In [35]:
```
acc = accuracy_score(y_test, y_pred)
acc
```

Out[35]:
```
0.5555555555555556
```

Confusion Matrix and Classification Report

In [36]:
```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
Confusion Matrix:
[[54 42]
 [46 56]]

Classification Report:
              precision    recall  f1-score   support

          No       0.54      0.56      0.55        96
         Yes       0.57      0.55      0.56       102

    accuracy                           0.56       198
   macro avg       0.56      0.56      0.56       198
weighted avg       0.56      0.56      0.56       198
```

In [ ]:

ROC-AUC

The Receiver Operating Characteristic (ROC) curve is used to evaluate the performance of a classification model at different classification thresholds. The Area Under the Curve (AUC) provides a single metric that summarizes the model's ability to distinguish between the classes. A higher AUC score indicates better classification performance.

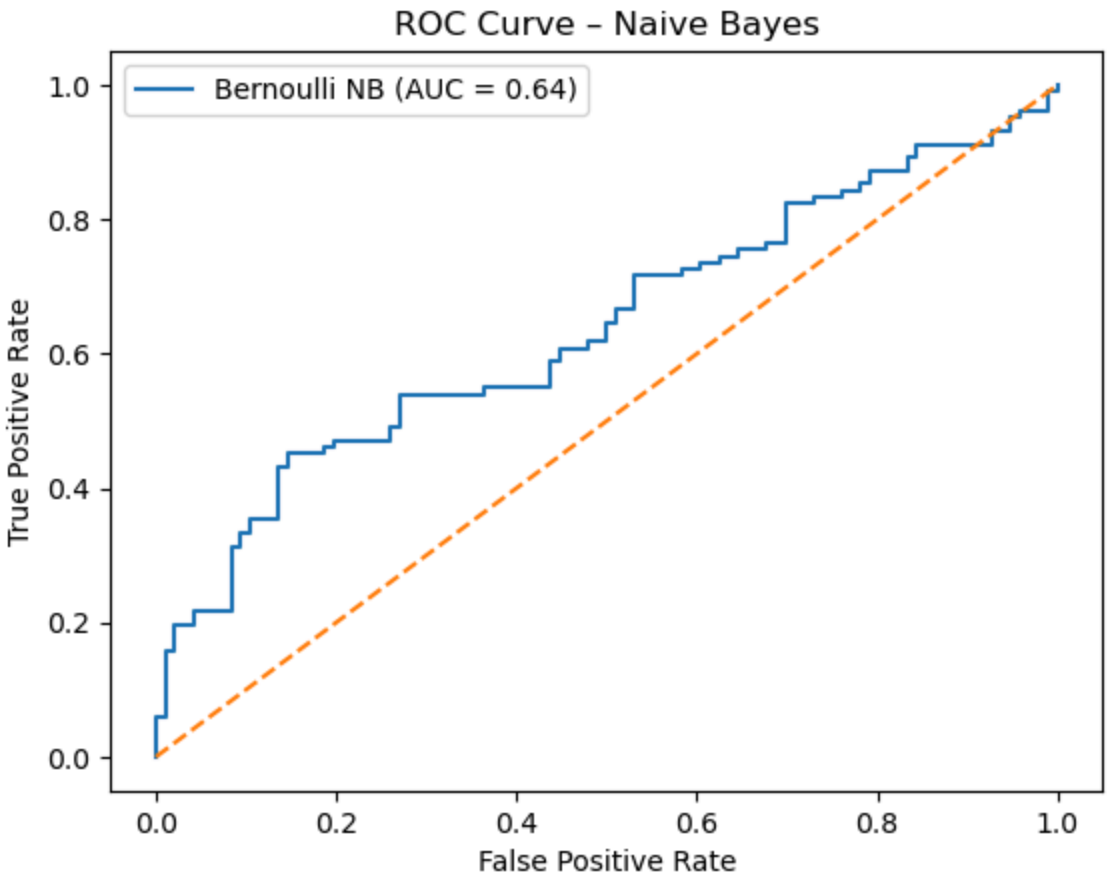
In [37]:
```
from sklearn.metrics import roc_curve, roc_auc_score
```

In [38]:
```
y_prob = nb.predict_proba(x_test)[:, 1]
roc_auc = roc_auc_score(y_test, y_prob)
roc_auc
```

Out[38]:
```
np.float64(0.6371527777777778)
```

In [39]:
```
fpr, tpr, thresholds = roc_curve(y_test, y_prob, pos_label='Yes')

plt.figure()
plt.plot(fpr, tpr, label=f'Bernoulli NB (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve – Naive Bayes')
plt.legend()
plt.show()
```



In [ ]:

RandomizedSearchCV

RandomizedSearchCV is used to optimize the hyperparameters of the Bernoulli Naive Bayes model by randomly sampling combinations of parameters from a predefined distribution.

In [40]:
```
nb = BernoulliNB()
param_dist = {
    'alpha': np.linspace(0.01, 2.0, 10),
    'fit_prior': [True, False]}

random_search = RandomizedSearchCV(estimator=nb,param_distributions=param_dist
                                   n_iter=5,cv=3,scoring='roc_auc',n_jobs=-1,r
random_search.fit(x_train, y_train)

print("Best Parameters (Random Search):", random_search.best_params_)
print("Best ROC-AUC:", random_search.best_score_)
```

```
Best Parameters (Random Search): {'fit_prior': False, 'alpha': np.float64(1.778
888888888889)}
Best ROC-AUC: 0.5741676087424635
```

In [ ]:

GridSearchCV

GridSearchCV tests multiple combinations of hyperparameters to find the best K value and distance metric.

In [41]:
```
best_alpha = random_search.best_params_['alpha']

param_grid = {
    'alpha': [best_alpha - 0.1, best_alpha, best_alpha + 0.1],
    'fit_prior': [True, False]}

grid_search = GridSearchCV(estimator=BernoulliNB(),param_grid=param_grid,cv=3,
grid_search.fit(x_train, y_train)

print("Best Parameters (Grid Search):", grid_search.best_params_)
print("Best ROC-AUC:", grid_search.best_score_)
```

```
Best Parameters (Grid Search): {'alpha': np.float64(1.878888888888889), 'fit_pr
ior': True}
Best ROC-AUC: 0.5742827513398411
```

In [42]:
```
best_nb = grid_search.best_estimator_

y_pred_best = best_nb.predict(x_test)
y_prob_best = best_nb.predict_proba(x_test)[:, 1]
```

In [43]:
```
print("Final Accuracy:", accuracy_score(y_test, y_pred_best))
print("Final ROC-AUC:", roc_auc_score(y_test, y_prob_best))
print(classification_report(y_test, y_pred_best))
```

```
Final Accuracy: 0.5606060606060606
Final ROC-AUC: 0.6380718954248366
              precision    recall  f1-score   support

          No       0.55      0.56      0.55        96
         Yes       0.58      0.56      0.57       102

    accuracy                           0.56       198
   macro avg       0.56      0.56      0.56       198
weighted avg       0.56      0.56      0.56       198
```

In [ ]:

In [ ]:

Improving Model Performance with Advanced Algorithms

In [ ]:

In [49]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifie

from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# Encode target
le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)

# Evaluation function
def evaluate(y_true, y_pred):
    return (
        accuracy_score(y_true, y_pred),
        precision_score(y_true, y_pred, average='weighted'),
        recall_score(y_true, y_pred, average='weighted'),
        f1_score(y_true, y_pred, average='weighted')
    )

results = []

# Logistic Regression
lr = GridSearchCV(
    LogisticRegression(max_iter=1000),
    {"C": [0.1, 1, 10]},
    cv=2
)
lr.fit(x_train, y_train)
y_pred = lr.predict(x_test)
results.append(["Logistic Regression", *evaluate(y_test, y_pred)])

# Decision Tree
dt = GridSearchCV(
    DecisionTreeClassifier(),
    {"max_depth": [3, 5, 10]},
    cv=2
)
dt.fit(x_train, y_train)
```

```python
y_pred = dt.predict(x_test)
results.append(["Decision Tree", *evaluate(y_test, y_pred)])

# Random Forest
rf = GridSearchCV(
    RandomForestClassifier(),
    {"n_estimators": [100, 200]},
    cv=2
)
rf.fit(x_train, y_train)
y_pred = rf.predict(x_test)
results.append(["Random Forest", *evaluate(y_test, y_pred)])

# Gradient Boosting
gb = GridSearchCV(
    GradientBoostingClassifier(),
    {"n_estimators": [100, 200], "learning_rate": [0.05, 0.1]},
    cv=2
)
gb.fit(x_train, y_train)
y_pred = gb.predict(x_test)
results.append(["Gradient Boosting", *evaluate(y_test, y_pred)])

# AdaBoost
ab = GridSearchCV(
    AdaBoostClassifier(),
    {"n_estimators": [50, 100]},
    cv=2
)
ab.fit(x_train, y_train)
y_pred = ab.predict(x_test)
results.append(["AdaBoost", *evaluate(y_test, y_pred)])

# XGBoost
xgb = GridSearchCV(
    XGBClassifier(use_label_encoder=False, eval_metric='mlogloss'),
    {"n_estimators": [100, 200], "learning_rate": [0.05, 0.1]},
    cv=2
)
xgb.fit(x_train, y_train)
y_pred = xgb.predict(x_test)
results.append(["XGBoost", *evaluate(y_test, y_pred)])

# LightGBM
lgb = GridSearchCV(
    LGBMClassifier(verbose=-1),
    {"n_estimators": [100, 200], "learning_rate": [0.05, 0.1]},
    cv=2
)
lgb.fit(x_train, y_train)
y_pred = lgb.predict(x_test)
results.append(["LightGBM", *evaluate(y_test, y_pred)])
```

```
# CatBoost
cb = GridSearchCV(
    CatBoostClassifier(verbose=0),
    {"iterations": [100, 200], "learning_rate": [0.05, 0.1]},
    cv=2
)
cb.fit(x_train, y_train)
y_pred = cb.predict(x_test)
results.append(["CatBoost", *evaluate(y_test, y_pred)])


# TABLE
columns = ["Model", "Accuracy", "Precision", "Recall", "F1 Score"]

df = pd.DataFrame(results, columns=columns)
df = df.sort_values(by="Accuracy", ascending=False)

display(df)
```

	Model	Accuracy	Precision	Recall	F1 Score
6	LightGBM	0.853535	0.854662	0.853535	0.853554
2	Random Forest	0.833333	0.833932	0.833333	0.833099
5	XGBoost	0.813131	0.815100	0.813131	0.813098
7	CatBoost	0.813131	0.813647	0.813131	0.812868
3	Gradient Boosting	0.752525	0.752984	0.752525	0.752582
1	Decision Tree	0.646465	0.654854	0.646465	0.643927
0	Logistic Regression	0.616162	0.616162	0.616162	0.616162
4	AdaBoost	0.616162	0.616465	0.616162	0.614029

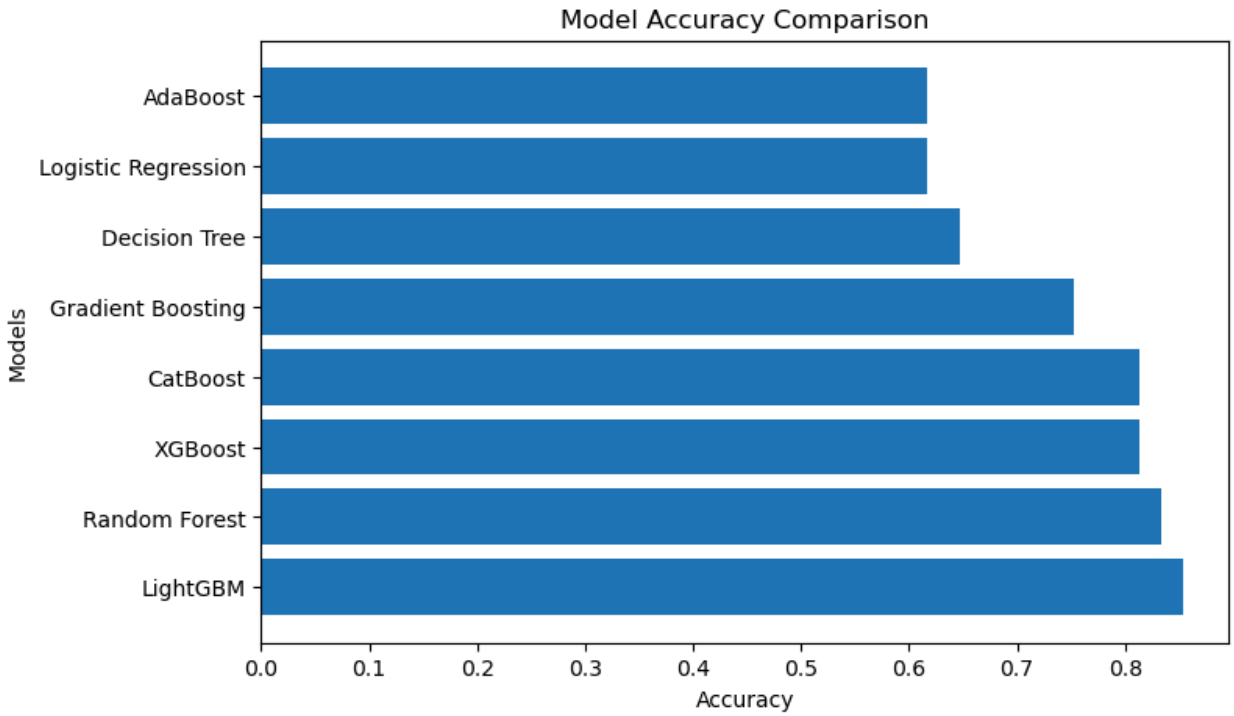
In [ ]:

In [52]:
```
plt.figure(figsize=(8,5))

plt.barh(df["Model"], df["Accuracy"])

plt.xlabel("Accuracy")
plt.ylabel("Models")
plt.title("Model Accuracy Comparison")

plt.show()
```



Boosting models (LightGBM, Random Forest, XGBoost, CatBoost) achieve the highest accuracy, while simpler models like Logistic Regression and AdaBoost perform comparatively lower

In [ ]:

In [ ]: